B1 & B2 — DESIGN AND ANALYSIS OF ALGORITHMS (DAA) PROJECT

Graph-Based Financial Maximum Gain Window

Dynamic Asset Switching on a Layered Directed Acyclic Graph (DAG) — Algorithm Design & Analysis Document

TEAM MEMBERS

- Shashwat Kashyap (Roll No: 590026643)
- Kushagra Ranjan (Roll No: 590025702)
- Shivang Raturi (Roll No: 590022331)
- Parneet Kaur (Roll No: 590022262)
- Ujjaiti Kumari Singh (Roll No: 590022882)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING | UNIVERSITY OF PETROLEUM AND ENERGY STUDIES

EXECUTIVE SUMMARY

Property	Description
Problem	Find one continuous time window and asset sequence with maximum net gain.
Required Method	Graph Algorithms — maximum-weight path relaxation on a layered DAG.
Core State	`(time, asset, switches used)`
Primary Output	Window, asset path, net gain, switch count, and detailed explanation.

1. PROBLEM DEFINITION

The Financial Maximum Gain Window problem is defined as an historical optimization problem. Given returns for multiple financial assets over ordered time periods, the system must select one continuous investment window and decide which single asset to hold in each period so that the total net gain is maximized. Switching from one asset to another is allowed but incurs a cost and is limited by a maximum switch budget.

Element	Definition
User	A financial analyst or investor studying historical opportunities.
Objective	Maximize cumulative return minus asset-switching costs.
Hard Constraints	One asset at a time; chronological movement; one continuous window; at most K switches; unavailable asset states cannot be selected.
Infeasible Result	If the optimum is non-positive, report that no profitable window exists rather than forcing an investment.

2. FORMAL MODEL

Let N be the number of ordered time periods and M the number of assets. Let $r(t, a)$ denote the additive gain or return produced by asset a during period t. Let $C(a, b)$ be the cost of switching from asset a to asset b, where $C(a, a) = 0$. Let K be the maximum number of asset switches permitted.

Input: $R = \{r(t, a) \mid 1 \le t \le N, 1 \le a \le M\}$, $C(a, b)$, K

The algorithm chooses a start time s, an end time e, and an asset sequence $a_s, \ldots, a_e$. The objective is to:

> **Maximize:** $G = \sum_{t=s..e} r(t, a_t) - \sum_{i=s+1..e} C(a_{i-1}, a_i)$
> **Subject to:** $1 \le s \le e \le N$, and (number of switches) $\le K$

The use of additive gains keeps the optimization transparent. If percentage returns and compounding are later required, the gain model must be changed rather than silently treating percentages as additive currency amounts.

3. WHY A GRAPH MODEL?

Kadane's algorithm finds a maximum-sum contiguous window in one linear sequence. Our problem has several possible predecessor choices at each time because the investor may remain in the current asset or switch from another asset. This naturally forms a layered directed acyclic graph (DAG).

4. LAYERED DAG CONSTRUCTION

A graph vertex represents a state (t, a, k): time t, currently selected asset a, and k switches already used. All edges advance from time t to $t+1$, so cycles are impossible.

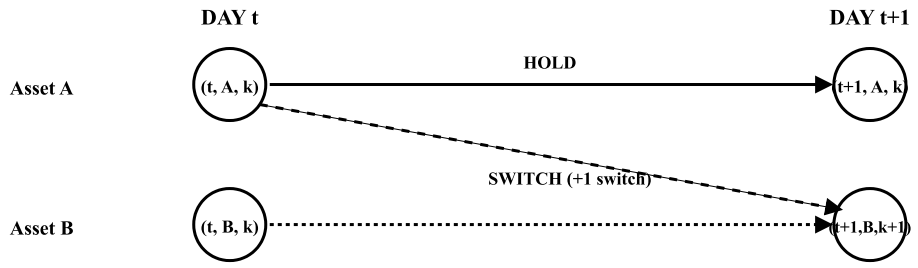


Figure 1. Conceptual layered DAG transitions. Horizontal transitions continue holding an asset; diagonal transitions switch assets and consume one switch.

Transition	From State	To State	Gain Contribution
Start	Outside window	$(t, a, 0)$	$+r(t, a)$
Hold	$(t-1, a, k)$	(t, a, k)	$+r(t, a)$
Switch	$(t-1, b, k-1)$	(t, a, k)	$+r(t, a) - C(b, a)$

5. PROPOSED ALGORITHM

5.1 Dynamic Programming State

Define `DP[t][a][k]` as the maximum net gain of any continuous window that ends at time t, holds asset a at time t, and has used exactly k switches. A predecessor table stores the previous state so that the selected window can be reconstructed.

5.2 Recurrence

For each state, there are three possibilities: start a new window at the current period, continue holding the same asset, or switch from a different asset.

- **Start:** $r(t, a)$
- **Hold:** $DP[t-1][a][k] + r(t, a)$
- **Switch:** $DP[t-1][b][k-1] + r(t, a) - C(b, a)$ for $b \neq a$

General Recurrence Relation:

```
DP[t][a][k] = max( Start (only when k=0), Hold, max_{b≠a}(Switch) )
```

5.3 Graph Interpretation

The recurrence is equivalent to maximum-weight relaxation in topological order. Because every edge moves forward by one time layer, chronological order is already a valid topological ordering. Each DP update is a relaxation of incoming DAG edges.

5.4 Pseudocode

```
MAX-GAIN-WINDOW(R, C, N, M, K):
    initialize DP[N][M][K+1] = -infinity
    initialize parent[N][M][K+1] = NONE
    best = 0, bestState = NONE

    for t = 0 to N-1:
        for a = 0 to M-1:
            if asset a is unavailable at t: continue

            // Start a new window
            if R[t][a] > DP[t][a][0]:
                DP[t][a][0] = R[t][a]
                parent[t][a][0] = START

            if t > 0:
                for k = 0 to K:
                    // Hold same asset
                    relax DP[t][a][k] from DP[t-1][a][k] + R[t][a]

                    // Switch from another asset
                    if k > 0:
                        for b = 0 to M-1, b != a:
                            candidate = DP[t-1][b][k-1] + R[t][a] - C[b][a]
                            relax DP[t][a][k] using candidate

            update best and bestState

    if best <= 0: return NO_PROFITABLE_WINDOW
    return RECONSTRUCT(parent, bestState)
```

6. WORKED EXAMPLE AND TRACE

Consider four time periods, two assets, a switching cost of 2 units, and at most one switch ($K=1$).

Day	Asset A	Asset B
1	-3	+2
2	**+7**	-2
3	**+5**	+3
4	-6	**+8**

Execution Steps:

- **Day 2:** START A → Gain = 7
- **Day 3:** HOLD A → Gain = 7 + 5 = 12
- **Day 4:** SWITCH A → B (Cost = 2) → Gain = 12 + 8 − 2 = **18**

7. CORRECTNESS ARGUMENT

Theorem: For every valid state *(t, a, k)*, `DP[t][a][k]` equals the maximum net gain of any feasible continuous window ending at time *t* in asset *a* after exactly *k* switches. Consequently, the maximum DP value is the globally optimal feasible gain window.

Proof by induction over time layers:

- **Base case:** A window ending in the first processed time layer can only start at that layer. The algorithm assigns its return directly, so the state is optimal.
- **Inductive hypothesis:** Assume every feasible state at time *t-1* stores its optimal value.
- **Inductive step:** Any feasible optimal window ending at *(t, a, k)* must be exactly one of three cases: it starts at *t*; it held asset *a* at *t-1* and continues holding; or it held some different asset *b* at *t-1* and switches to *a*, consuming one switch and paying *C(b, a)*. The recurrence evaluates every feasible predecessor of these forms and selects the maximum. By the inductive hypothesis, each predecessor value is already optimal. Therefore `DP[t][a][k]` is optimal. **(Q.E.D.)**

8. COMPLEXITY ANALYSIS

- **Time Complexity:** $O(N \cdot M^2 \cdot (K+1))$
- **DP Value Memory (rolling layers):** $O(M \cdot (K+1))$
- **Reconstruction Metadata:** $O(N \cdot M \cdot (K+1))$

9. BASELINE ALGORITHMS

Method	Purpose	Strength	Limitation
Brute-force enumeration	Exact oracle on small instances	Independently verifies optimum	Combinatorial growth
Kadane per asset	Classical window baseline	*O(N)* per asset	Cannot switch assets or charge switch costs
Proposed Layered-DAG DP	Full constrained problem	Exact, reconstructable, graph-based	More state space than Kadane

10. DATA HANDLING & EDGE CASES

Case	Treatment
Empty Input	Return `EMPTY_DATASET`; no graph is constructed.
All Non-Positive Gains	Return `NO_PROFITABLE_WINDOW`.
Tie Between Windows	Higher gain, then fewer switches, then earlier start, then shorter window.
Missing/Invalid Return	Mark asset-time state unavailable; skip incoming/outgoing transitions.

11. IMPLEMENTATION MODULES

- `parseDataset()`: Read CSV/JSON records.
- `cleanDataset()`: Handle invalid data.
- `solveGraphDP()`: Run DAG relaxation.
- `reconstructPath()`: Recover strategy.